AutonomousSoGoodAgent.js

AutonomousSoGoodAgent.js

What it does:

1. Listens to PostSubmitted events from SoGoodFeed.sol

2. Pulls the IPFS content using contentHash

3. Sends it to OpenAI for scoring + explanation

4. Uploads full result to IPFS (JSON)

5. Calls recordTip(postId, user, scoreAmount) using ethers.js

--

Result:

Runs 24/7 — you can deploy this as a Cloud Function, on a Raspberry Pi, or serverless node

Automatically evaluates, scores, IPFS-uploads, and records tips

No need for user to do anything but post

```
import { ethers } from "ethers";
import { Configuration, OpenAIApi } from "openai";
import axios from "axios";
import fs from "fs";
import dotenv from "dotenv";
dotenv.config();

// Smart contract + provider setup
const provider = new ethers.providers.WebSocketProvider(process.env.ALCHEMY_WS_URL);
const wallet = new ethers.Wallet(process.env.SIGNER_PRIVATE_KEY, provider);
const feedAddress = process.env.SOGOOD_FEED_ADDRESS;
const feedABI = JSON.parse(fs.readFileSync("../abis/SoGoodFeed.json")); // your ABI file
const feed = new ethers.Contract(feedAddress, feedABI, wallet);

// OpenAI config
const openai = new OpenAIApi(new Configuration({
  apiKey: process.env.OPENAI_API_KEY,
}));

// Listen for new posts
feed.on("PostSubmitted", async (postId, author, contentHash, metadata) => {
  console.log(`New post detected! Post ID: ${postId}`);

  try {
    // 1. Fetch content from IPFS
    const ipfsURL = `https://gateway.pinata.cloud/ipfs/${contentHash}`;
    const { data: content } = await axios.get(ipfsURL);

    // 2. Score with GPT
    const prompt = `You are a decentralized AI evaluator. Score this social content (0–100) for transparency, insight, and real-world utility:\n\n"${content}"\n\nRespond in JSON: { "score": number, "reason": string }`;
    const gptRes = await openai.createChatCompletion({
      model: "gpt-4",
      messages: [{ role: "user", content: prompt }],
    });

    const scoreData = JSON.parse(gptRes.data.choices[0].message.content);
    const score = scoreData.score;

    // 3. Prepare metadata and upload to IPFS
    const result = {
      postId: postId.toString(),
      content,
      score,
      explanation: scoreData.reason,
      author,
      timestamp: new Date().toISOString(),
    };

    fs.writeFileSync("tip_metadata.json", JSON.stringify(result, null, 2));

    const ipfsRes = await axios.post("https://api.pinata.cloud/pinning/pinFileToIPFS",
      fs.createReadStream("tip_metadata.json"),
      {
        maxBodyLength: Infinity,
        headers: {
          "Content-Type": `multipart/form-data`,
          pinata_api_key: process.env.PINATA_API_KEY,
          pinata_secret_api_key: process.env.PINATA_SECRET,
        },
      }
    );

    const tipIPFS = `ipfs://${ipfsRes.data.IpfsHash}`;
    console.log(`Scored: ${score} | Metadata IPFS: ${tipIPFS}`);

    // 4. Call recordTip
    const tx = await feed.recordTip(postId, author, score);
    await tx.wait();
    console.log(`Tip recorded onchain: ${tx.hash}`);
  } catch (err) {
    console.error("Error processing post:", err.message);
  }
});
```

.env Sample

```
ALCHEMY_WS_URL=wss://eth-sepolia.g.alchemy.com/v2/YOUR_KEY
SIGNER_PRIVATE_KEY=0xabc123...
SOGOOD_FEED_ADDRESS=0xSoGoodFeed...
OPENAI_API_KEY=sk-...
PINATA_API_KEY=...
PINATA_SECRET=...
```

--

Notein
Scan the QR code to download Notein